

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

27 - Floating Point

Introduction

- Programming languages support numbers with **fractions**
 - Called **reals** in mathematics
- Examples of reals:

$3.14159265..._{\text{ten}}$ (π)

$2.71828..._{\text{ten}}$ (e)

0.000000001_{ten} or $1.0_{\text{ten}} \times 10^{-9}$ (seconds in a nanosecond)

$3,155,760,000_{\text{ten}}$ or $3.15576_{\text{ten}} \times 10^9$ (seconds in a typical century)

- Last case, number is bigger than we could represent with a 32-bit signed integer

Scientific Notation

- Alternative notation for last two numbers is called **scientific notation**
- Has a single digit to the left of the decimal point

$3.14159265..._{\text{ten}}$ (π)

$2.71828..._{\text{ten}}$ (e)

0.000000001_{ten} or $1.0_{\text{ten}} \times 10^{-9}$ (seconds in a nanosecond)

$3,155,760,000_{\text{ten}}$ or $3.15576_{\text{ten}} \times 10^9$ (seconds in a typical century)

Normalized Scientific Notation

- A number that has **no leading 0s** is called a **normalized** number
- For example
 - $1.0_{\text{ten}} \times 10^{-9}$ is in normalized scientific notation
 - $0.1_{\text{ten}} \times 10^{-8}$ and $10.0_{\text{ten}} \times 10^{-10}$ are not

Binary Scientific Notation

- We can also show binary numbers in scientific notation:

$$1.0_{\text{two}} \times 2^{-1}$$

- To keep binary number in normalized form, we can increase or decrease base by exactly the number of bits the number must be shifted to have one nonzero digit to left of decimal point
- Base of 2 fulfills our need
- Need a new name for decimal point
 - Binary point

Floating Point

- Computer arithmetic that supports such numbers is called *Floating Point* (FP)
 - Because it represents numbers in which binary point is not fixed, as it is for integers
- Programming language uses name `float` for such numbers
- As in scientific notation, numbers are represented as a single nonzero digit to left of binary point

Floating Point Normalized Form

$$1.\text{xxxxxxxx}_{\text{two}} \times 2^{\text{yyyy}}$$

- Although computer represents exponent in base 2 as well as rest of the number, to simplify notation we show exponent in decimal

Advantages

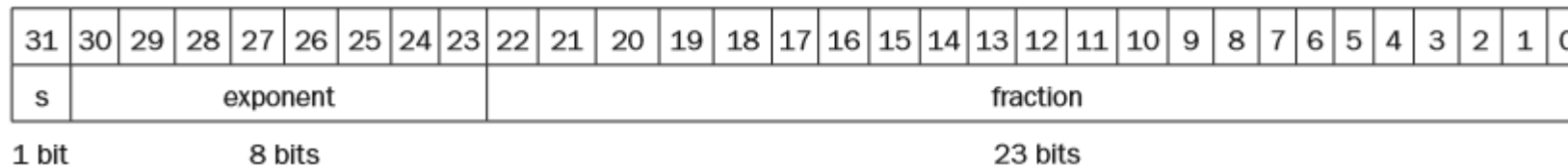
- Standard scientific notation for reals in normalized form offers three advantages
 1. Simplifies exchange of data that includes FP numbers
 2. Simplifies FP arithmetic algorithms to know that numbers will always be in this form
 3. Increases accuracy of numbers that can be stored in a word, since unnecessary leading 0s are replaced by real digits to the right of the binary point

Floating Point Representation

- A designer of FP representation must find a compromise between size of **fraction** and size of **exponent**
 - Fixed word size means: must take bit from one to add bit to the other
- This tradeoff is between **precision** and **range**
 - Increasing size of fraction enhances precision of the fraction
 - Increasing size of exponent increases range of the numbers
- Good design demands good compromise

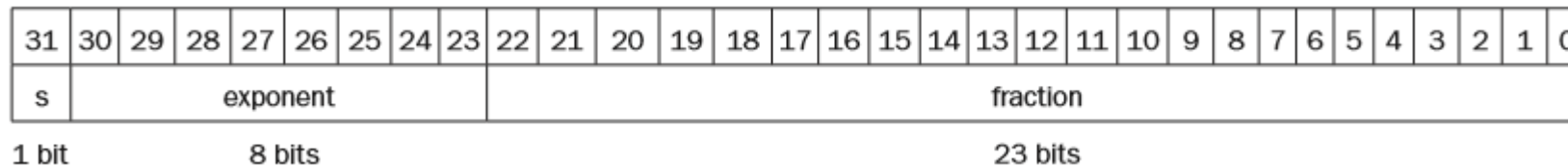
Floating Point Representation

- FP numbers are usually multiple of the size of a word



Floating Point Representation

- FP numbers are usually multiple of the size of a word



- Sign (s) of FP number (1 meaning negative)
- Exponent (E) is value of 8-bit exponent field (including sign of exponent)
- Fraction (F) is 23-bit number
- Representation is sign and magnitude
 - since sign is a separate bit from the rest of the number

Floating Point Format (Single Precision)

- In general, FP numbers are of the form

$$(-1)^S \times F \times 2^E$$

- S involves sign (0: positive, 1: negative)
- F involves value in fraction field
- E involves the value in the exponent field

Range

- Chosen sizes give computer arithmetic an extraordinary range
 - as small as $2.0_{\text{ten}} \times 10^{-38}$
 - as large as $2.0_{\text{ten}} \times 10^{38}$

Overflow and Underflow

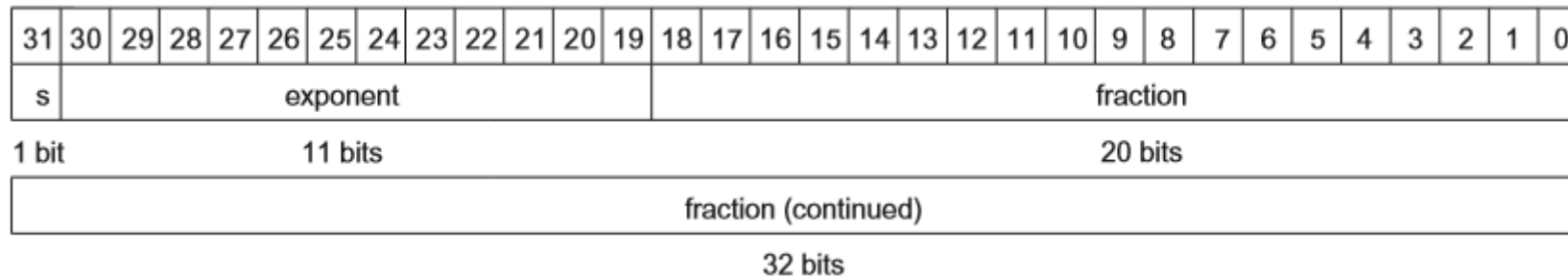
- Overflow means that exponent is too large to be represented in exponent field
- Underflow occurs when negative exponent is too large to fit in exponent field

Double Precision

- One way to reduce chances of underflow or overflow is to offer another format that has a larger exponent
- It is called **double**
- Operations on doubles are called **double precision** FP arithmetic

Double Precision Format

- Representation of a double precision FP number takes two words



- S** is still **sign** of the number
- E** is value of the 11-bit **exponent** field
- F** is 52-bit number in **fraction** field

Range

- MIPS double precision allows numbers
 - as small as $2.0_{\text{ten}} \times 10^{-308}$
 - as large as $2.0_{\text{ten}} \times 10^{308}$
- Although double precision does increase exponent range, its primary advantage is its greater precision because of the much larger fraction

Standard

- These formats are part of *IEEE 754 floating-point standard*, found in virtually every computer invented since 1980
- This standard has greatly improved both the ease of porting floating-point programs and the quality of computer arithmetic

Significand and Fraction

- To pack even more bits into the significand, IEEE 754 makes the leading 1-bit of normalized binary numbers **implicit**
- Hence, number is actually
 - 24 bits long in single precision (implied 1 and a 23-bit fraction)
 - 53 bits long in double precision (1 + 52)
- To be precise, use the term
 - **Significand** to represent the 24- or 53-bit number (1 + fraction)
 - **Fraction** when we mean the 23- or 52-bit number

Numbers Representations

- Number Zero (0)
 - Since 0 has no leading 1, it is given reserved exponent value 0
- So that the HW won't attach a leading 1 to it
- Thus 00 ... 00_{two} represents 0

Numbers Representations

- Rest of numbers uses form from before with hidden 1 added:

$$(-1)^S \times (1 + \text{Fraction}) \times 2^E$$

where bits of fraction represent a number between 0 and 1
and E specifies value in exponent field

- If we number bits of fraction LTR: $s_1, s_2, s_3, \dots$, then value is

$$(-1)^S \times (1 + (s_1 \times 2^{-1}) + (s_2 \times 2^{-2}) + (s_3 \times 2^{-3}) + (s_4 \times 2^{-4}) + \dots) \times 2^E$$

Encoding of Floating-point Numbers

- Features of IEEE 754 are special symbols to represent unusual events; For example
 - Instead of interrupting on a **divide by 0**, SW can set result to a bit pattern representing $+\infty$ or $-\infty$
 - Largest exponent is reserved for these special symbols
 - When programmer prints results, program will print **infinity** symbol
 - Symbol for result of invalid operations, such as **0/0** or $\infty - \infty$
 - Symbol is **NaN**, for *Not a Number*
 - To allow programmers to postpone some tests and decisions to a later time in program

Encoding of Floating-point Numbers

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

Encoding of Floating-point Numbers

- Designers of IEEE 754 also wanted FP representation that could be easily processed by integer comparisons, especially for sorting
- This is why sign is in MSB
 - Allowing a quick test of $<$, $>$, or $= 0$

Biased Notation

- Negative exponents pose a challenge
- Desirable notation must therefore represent
 - most negative exponent as $00 \dots 00_{\text{two}}$ and
 - most positive as $11 \dots 11_{\text{two}}$
- This convention is called *biased notation*
 - with *bias* being the number subtracted from the normal, unsigned representation to determine the real value

Biased Notation

- IEEE 754 uses a **bias** of **127** for **single precision**
 - Exponent of -1 is represented by $-1 + 127_{\text{ten}}$, or $126_{\text{ten}} = 0111\ 1110_{\text{two}}$
 - Exponent of 1 is represented by $1 + 127_{\text{ten}}$, or $128_{\text{ten}} = 1000\ 0000_{\text{two}}$
- Exponent **bias** for **double precision** is **1023**

Biased Notation

- Biased exponent means that the value represented by a FP number is

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent} + \text{Bias}}$$

- Range of single precision numbers is then
 - From as small as $\pm 1.0000000000000000000000000000_{\text{two}} \times 2^{-126}$
 - to as large as $\pm 1.1111111111111111111111111111_{\text{two}} \times 2^{+127}$

Example

- Show the IEEE 754 binary representation of the number -0.75_{ten} in single and double precision.

- Number -0.75_{ten} is also

$$-3/4_{\text{ten}} \text{ or } -3/2^2_{\text{ten}}$$

- It is also represented by binary fraction

$$-11_{\text{two}} / 2^2_{\text{ten}} \text{ or } -0.11_{\text{two}}$$

Example (cont.)

- In scientific notation, the value is

and in normalized scientific notation, it is

Example (cont.)

- In scientific notation, the value is

$$-0.11_{\text{two}} \times 2^0$$

and in normalized scientific notation, it is

$$-1.1_{\text{two}} \times 2^{-1}$$

Example (cont.)

- General representation for a single precision number is

$$(-1)^s \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - 127)}$$

- Subtracting the bias 127 from the exponent yields

$$(-1)^1 \times (1 + .1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 000_{\text{two}}) \times 2^{(126 - 127)}$$

$$\text{— Exponent} = 126_{\text{ten}} = 01111110_{\text{two}}$$

Example (cont.)

- General representation for a single precision number is

$$(-1)^s \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - 127)}$$

- Subtracting the bias 127 from the exponent yields

$$(-1)^1 \times (1 + .1000\ 0000\ 0000\ 0000\ 0000\ 000_{\text{two}}) \times 2^{(126 - 127)}$$

– Exponent = $126_{\text{ten}} = 01111110_{\text{two}}$

- Single precision binary representation of -0.75_{ten} is then

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

1 bit

8 bits

23 bits

Example (cont.)

- General representation for a single precision number is

$$(-1)^s \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - 127)}$$

- Subtracting the bias 127 from the exponent yields

$$(-1)^1 \times (1 + .1000\ 0000\ 0000\ 0000\ 0000\ 000_{\text{two}}) \times 2^{(126 - 127)}$$

– Exponent = $126_{\text{ten}} = 01111110_{\text{two}}$

- Single precision binary representation of -0.75_{ten} is then

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

1 bit

8 bits

23 bits

Example (cont.)

- Double precision representation is

$$(-1)^1 \times (1 + .1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{\text{two}}) \times 2^{(1022-1023)}$$

Converting Binary to Decimal Floating Point

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1 bit		11 bits										20 bits																			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
32 bits																															

Converting Binary to Decimal Floating Point

- Single or double precision number can be converted to decimal
- Number is divided into:
 - Sign
 - Exponent
 - Fraction

Example

- What decimal number is represented by single precision float?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

Example

- What decimal number is represented by single precision float?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

- Sign: 1, Exponent: 129, and Fraction: $1 \times 2^{-2} = 1/4 = 0.25$

Example

- What decimal number is represented by single precision float?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

- Sign: 1, Exponent: 129, and Fraction: $1 \times 2^{-2} = 1/4 = 0.25$
- Using the basic equation

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} = (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)}$$

Example

- What decimal number is represented by single precision float?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

- Sign: 1, Exponent: 129, and Fraction: $1 \times 2^{-2} = 1/4 = 0.25$
- Using the basic equation

$$\begin{aligned} (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} &= (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)} \\ &= -1 \times 1.25 \times 2^2 \end{aligned}$$

Example

- What decimal number is represented by single precision float?

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

- Sign: 1, Exponent: 129, and Fraction: $1 \times 2^{-2} = 1/4 = 0.25$
- Using the basic equation

$$\begin{aligned} (-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} &= (-1)^1 \times (1 + 0.25) \times 2^{(129 - 127)} \\ &= -1 \times 1.25 \times 2^2 \\ &= -1.25 \times 4 \\ &= -5.0 \end{aligned}$$